The ML System Design Framework

Key Takeaways

Business metrics first, always

Most ML projects die because the team optimised an ML metric that did not connect to a business outcome. Before writing any model code, answer: what business metric should move, by how much, and how does my ML objective trace back to it?

The four requirements drive architecture

Reliability (silent failures), scalability (traffic, complexity, model count), maintainability (multiple contributors), and adaptability (shifting distributions) are not optional features. They are the structural load-bearing requirements of any ML system. Ignoring any one of them guarantees a costly rebuild.

The process is iterative and never-ending

The 6-step cycle (scope → data → model → deploy → monitor → analyse) never completes. Business analysis routinely sends you back to scoping. Monitoring findings trigger new data engineering. Design systems that expect change, not systems that assume stability.

Problem framing changes everything

The same business problem can be framed as regression or classification, coupled or decoupled objectives. The framing determines whether adding a new class requires retraining, whether product managers can tune weights without ML engineers, and whether the system is maintainable at scale. Spend time on framing before spending time on models.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. A music streaming company deploys a recommendation model. After six weeks, the model's offline NDCG score remains high, but user session lengths have dropped 12%. Using the business-to-ML metric framework from this session, explain what likely went wrong and what the team should measure instead.

Q2. A team wants to add a content safety filter to their social platform. A junior engineer proposes training one model that jointly predicts spam, hate speech, misinformation, and NSFW content using a combined loss. What is the key architectural argument against this approach, and what would you propose instead?

Q3. An e-commerce company wants to predict which users will churn in the next 30 days. Frame this as a complete ML problem: state the task type, input features (give 3 concrete examples), output, objective function, and one business metric that a successful model should improve.

Q4. Explain the difference between reliability and adaptability in ML systems. Why is it possible to have high reliability but low adaptability? Give a one-sentence example of each failure.

Design Exercises

Work through these independently or in pairs. Focus on precision: vague answers are not acceptable in system design.

Exercise 1: Translate a Business Problem into an ML Problem

An e-commerce company says: "Our customers abandon their shopping carts too often. We want to reduce cart abandonment by 15% this quarter."

(a) Frame this as a complete ML problem. State: the task type (regression/binary/multiclass/multilabel), the input features (list at least 4 concrete ones), the output, and the objective function (name the loss).

(b) Is this a regression or classification problem? Could it be framed either way? What are the trade-offs?

(c) What business metric would you track to know whether the deployed model actually helped? How does it connect to the ML metric?

(d) Identify one reliability concern and one adaptability concern for this system.

Exercise 2: Decoupling Objectives for Newsfeed Ranking

A social platform's newsfeed ranking system needs to satisfy three objectives simultaneously: (1) filter spam, (2) rank posts by quality, and (3) rank posts by user engagement. Currently, a single model handles all three with a combined weighted loss.

(a) Draw or describe the architecture of the decoupled approach. How many models are there, what does each predict, and how are their outputs combined?

(b) A product manager wants to increase the weight on quality relative to engagement for a weekend experiment. Under the current (combined) architecture, what does this require? Under the decoupled architecture, what does this require?

(c) What is one scenario where keeping the objectives coupled in a single model would be preferable? (Hint: think about data or latency constraints.)

Challenge Problem

Pick a Company. Apply the Full Framework.

Choose one of the following companies: **Spotify**, **Uber**, or **LinkedIn**. Identify **three distinct ML use cases** for that company. For each use case, write a structured paragraph covering:

(1) The **business metric** the use case is intended to improve (be specific — e.g., "7-day retention rate" not just "engagement")

(2) The **ML problem framing**: task type, input, output, objective function

(3) Which of the four requirements (reliability, scalability, maintainability, adaptability) is the *most critical* for this use case, and why

(4) One way the system could fail silently, and how you would detect it

Aim for approximately one page total. Precision and specificity are more valuable than length.

Further Reading

- **Chip Huyen — Designing Machine Learning Systems, Ch. 2** — O'Reilly, 2022 (course primary text)
  The direct source for today's session. The sections on "Framing ML Problems," "Decoupling Objectives," and "Mind vs. Data" are essential reading. Available via O'Reilly Learning.
- **Rich Sutton — The Bitter Lesson (2019)** — incompleteideas.net/IncIdeas/BitterLesson.html
  A 5-minute read on why computation and data beat hand-crafted algorithms over 70 years of AI research. Directly relevant to the mind vs. data debate.
- **Monica Rogati — The AI Hierarchy of Needs (2017)** — hackernoon.com/@mrogati
  The pyramid that argues AI is only as strong as its data foundation. Widely cited in industry; worth reading alongside Sutton for a balanced view.
- **Peter Norvig et al. — The Unreasonable Effectiveness of Data (2009)** — IEEE Intelligent Systems, Vol. 24
  The original "more data beats better algorithms" paper. Short and worth reading to understand where the Norvig quote comes from.

Online Resources

- **Chip Huyen's Blog** — huyenchip.com — practical ML system design essays directly tied to the book
- **Eugene Yan — Applied ML** — eugeneyan.com — real-world case studies from Amazon and other companies
- **Netflix Tech Blog** — netflixtechblog.com — the take-rate and recommendation system details come from here
- **Recsys Conference Papers** — recsys.acm.org — for deeper study of ranking and objective decoupling
- **The Batch (DeepLearning.AI)** — weekly newsletter with practical ML system examples
- **Evidently AI Blog** — evidentlyai.com — for the monitoring and adaptability side of today's session

Answers

A1. NDCG measures ranking quality on held-out data, but it does not capture whether users *acted* on the recommendations. What likely went wrong: the model optimised for historical click patterns that no longer reflect current user preferences (concept drift), or the recommendation set lacked diversity, leading to lower engagement over time. The team should measure a business-aligned proxy — session length, weekly active listening days, or a take-rate equivalent (plays per recommendation shown) — not just offline ranking metrics.

A2. The key argument: when objectives are combined in a single model, changing the relative weight of any objective requires full retraining. This is expensive, slow, and ties product decisions to ML engineering cycles. The decoupled approach trains separate models for spam, hate speech, misinformation, and NSFW detection. At serving time, scores from each model are combined using tunable weights. Product managers can adjust weights without touching any model. ML engineers can retrain individual models independently.

A3. Accept any well-reasoned answer. Example: binary classification (churn vs. no-churn). Inputs: days since last purchase, total spend in past 90 days, number of sessions in past 30 days (at least 3 required). Output: probability of churn within 30 days. Objective: binary cross-entropy (logistic loss). Business metric: 30-day customer retention rate. The model's churn probability should inversely correlate with observed retention — validate this empirically before and after deployment.

A4. Reliability means the system continues to produce correct predictions in the face of adversity (bad inputs, infrastructure failures). Adaptability means the system can evolve with shifting data distributions without service interruption. You can have high reliability but low adaptability: a highly reliable fraud detection model that runs 24/7 without errors but whose accuracy declines to 60% over 18 months as fraud patterns shift — it never crashes, but it stops being useful. You can also have high adaptability but low reliability: a system that retrains daily but has no input validation, so bad data periodically causes silent garbage predictions.

Course materials: priyanshsaxena.com/sst-ml-system-design **Instructor:** Priyansh Saxena